NAME:ROHAN RESHI

PRN:202401040034

BATCH:A1-6

GITHUB LINK: https://github.com/astarohan/AIML-LAB-PEC

LAB Assignment 4: Performance Evaluation of Regression Models

Aim

To implement and evaluate Simple Linear Regression, Multiple Linear Regression, Polynomial Regression, and Gradient Descent using different datasets.

Problem Statement

Regression algorithms predict continuous numerical values. This experiment implements different regression techniques, evaluates their prediction errors, and studies their performance using graphs and statistical metrics.

Objectives

1. Implement Simple Linear Regression with and without a regression library.
2. Implement Multiple Linear Regression with and without a regression library.
3. Implement Polynomial Regression.
4. Implement Gradient Descent from scratch.
5. Evaluate models using MAE, MSE, RMSE, and R^2.
6. Visualize the actual and predicted values and compare model performance.

Dataset Description

In this assignment, two different datasets were used to perform and compare regression techniques.

1. Advertising Spend vs Sales Dataset

A sample dataset was created containing two numerical columns: Advertising Spend and Sales. Advertising Spend is the independent variable (input), and Sales is the dependent variable (output). This dataset was used for Simple Linear Regression with a library and without a regression library.

2. Diabetes Dataset

The Diabetes dataset provided by Scikit-learn was used for Multiple Linear Regression, Polynomial Regression, Gradient Descent, and Locally Weighted Regression (LWR). It contains 442 observations, 10 numerical input features, and a continuous target representing a quantitative measure of disease progression.

- **Multiple Linear Regression:** Used all 10 input features to predict the target.

- **Polynomial Regression:** Used the BMI feature with a degree-2 polynomial to model a nonlinear relationship with the target.
- **Gradient Descent:** Used all 10 input features to learn regression coefficients iteratively.
- **Locally Weighted Regression:** Used all 10 input features and assigned higher weights to nearby training observations when making predictions.

The datasets were divided into training and testing sets to evaluate model performance. The models were assessed using MAE, MSE, RMSE, and R^2. These metrics helped measure prediction errors and determine how well each model explained variation in its target variable.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

print("Libraries imported successfully")
```

```
Libraries imported successfully
```

Theory: Libraries Used

NumPy is used for numerical calculations, Pandas for dataset handling, and Matplotlib for graphs. Scikit-learn provides regression models, data splitting, dataset loading, and evaluation metrics. For the manual implementations, NumPy performs the calculations without using a regression model class.

```
# Advertising Spend vs Sales dataset
advertising_df = pd.DataFrame({
    "AdvertisingSpend": [
        5, 8, 10, 12, 15, 18, 20, 22, 25, 28,
        30, 33, 35, 38, 40, 43, 45, 48, 50, 55
    ],
    "Sales": [
        12, 16, 18, 21, 25, 27, 30, 33, 35, 39,
        42, 44, 47, 50, 53, 56, 59, 62, 65, 71
    ]
})

display(advertising_df.head())

X_adv = advertising_df[["AdvertisingSpend"]]
y_adv = advertising_df["Sales"]

X_adv_train, X_adv_test, y_adv_train, y_adv_test = train_test_split(
    X_adv, y_adv, test_size=0.25, random_state=42
)

simple_model = LinearRegression()
simple_model.fit(X_adv_train, y_adv_train)

y_adv_pred = simple_model.predict(X_adv_test)

print("Slope:", simple_model.coef_[0])
print("Intercept:", simple_model.intercept_)
print(
```

```
    f"Regression Equation: Sales = "
    f"{simple_model.coef_[0]:.2f} × AdvertisingSpend + "
    f"{simple_model.intercept_:.2f}"
)
```

	AdvertisingSpend	Sales
0	5	12
1	8	16
2	10	18
3	12	21
4	15	25

```
Slope: 1.157798887631048
Intercept: 6.788846778559822
Regression Equation: Sales = 1.16 × AdvertisingSpend + 6.79
```

Theory: Simple Linear Regression Using a Library

Dataset: Advertising Spend vs Sales.

This experiment uses a small advertising dataset created in the notebook. Advertising expenditure is the independent variable, and sales is the dependent variable.

Simple Linear Regression predicts an output using one input feature. The model fits a straight line to the data.

Equation:

$(y = mx + c)$

Here, (y) is the predicted sales, (x) is advertising expenditure, (m) is the slope, and (c) is the intercept.

The Scikit-learn `LinearRegression` class calculates the best-fitting line. Model performance is evaluated using MAE, MSE, RMSE, and R^2.

```
mse_adv = mean_squared_error(y_adv_test, y_adv_pred)

print("MAE:", mean_absolute_error(y_adv_test, y_adv_pred))
print("MSE:", mse_adv)
print("RMSE:", np.sqrt(mse_adv))
print("R²:", r2_score(y_adv_test, y_adv_pred))
```

```
MAE: 0.4600580794408639
MSE: 0.2673259873966593
RMSE: 0.5170357699392367
R²: 0.9993488747384142
```

```
plt.figure(figsize=(8, 5))

plt.scatter(
    X_adv["AdvertisingSpend"], y_adv,
    color="red", label="Actual Data"
)

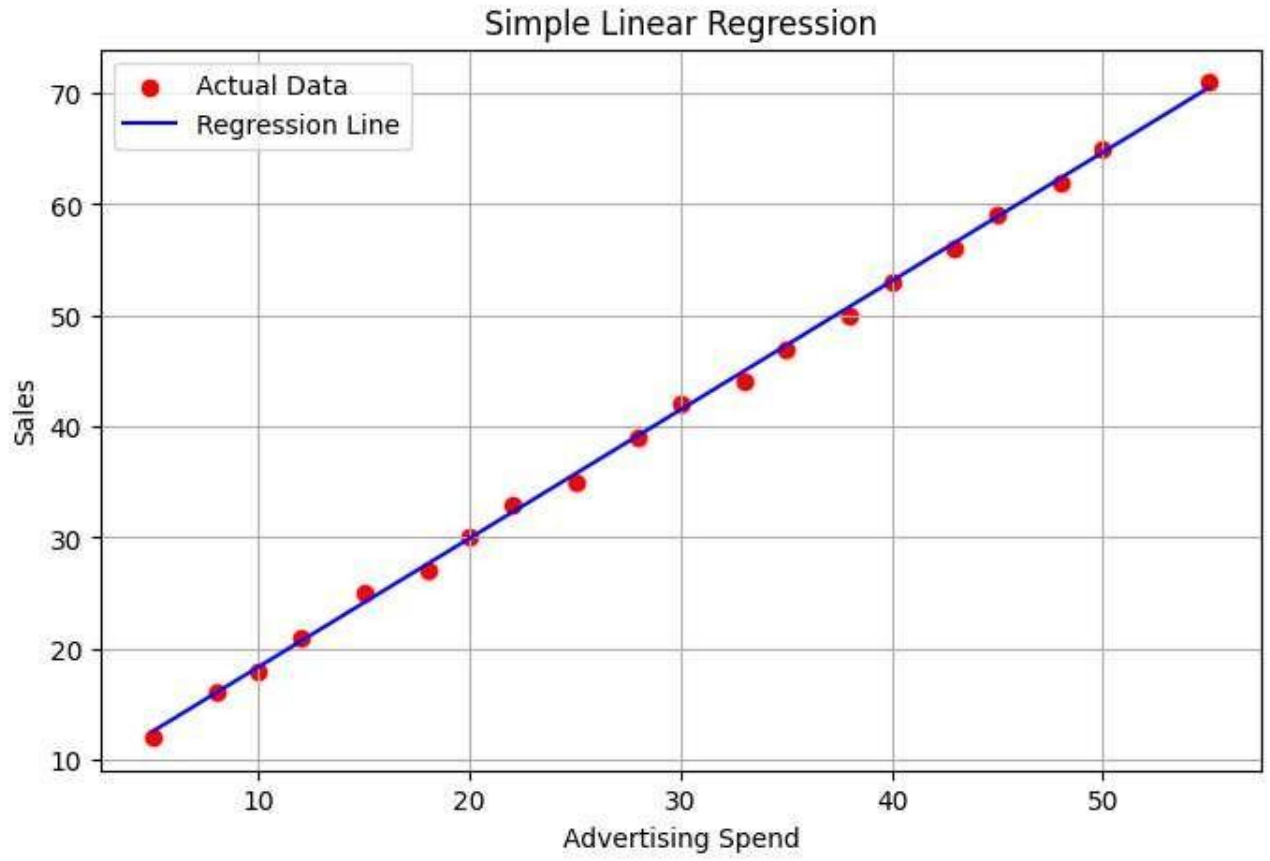
x_line = np.linspace(
    X_adv["AdvertisingSpend"].min(),
    X_adv["AdvertisingSpend"].max(),
    100
```

```
).reshape(-1, 1)

plt.plot(
    x_line, simple_model.predict(x_line),
    color="blue", label="Regression Line"
)

plt.xlabel("Advertising Spend")
plt.ylabel("Sales")
plt.title("Simple Linear Regression")
plt.legend()
plt.grid(True)
plt.show()
```

```
/usr/local/lib/python3.13/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does
  warnings.warn(
```




```
x = advertising_df["AdvertisingSpend"].to_numpy(dtype=float)
y = advertising_df["Sales"].to_numpy(dtype=float)

x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.25, random_state=42
)

x_mean = np.mean(x_train)
y_mean = np.mean(y_train)

slope_manual = np.sum(
    (x_train - x_mean) * (y_train - y_mean)
) / np.sum((x_train - x_mean) ** 2)

intercept_manual = y_mean - slope_manual * x_mean

y_adv_pred_manual = slope_manual * x_test + intercept_manual

print("Slope:", slope_manual)
print("Intercept:", intercept_manual)
```

```
print(
    f"Regression Equation: y = "
    f"{slope_manual:.2f}x + {intercept_manual:.2f}"
)
```

```
Slope: 1.157798887631048
Intercept: 6.788846778559822
Regression Equation: y = 1.16x + 6.79
```

Theory: Simple Linear Regression Without a Regression Library

Dataset: Advertising Spend vs Sales — the same dataset used in Section 1.

In this method, the slope and intercept are calculated manually using mathematical formulas. The Scikit-learn regression class is not used.

Slope:

$$m = \frac{\sum (x-\bar{x})(y-\bar{y})}{\sum (x-\bar{x})^2}$$

Intercept:

$$c = \bar{y} - m\bar{x}$$

Prediction:

$$y = mx + c$$

NumPy is used for arithmetic, while the regression calculations are implemented directly.

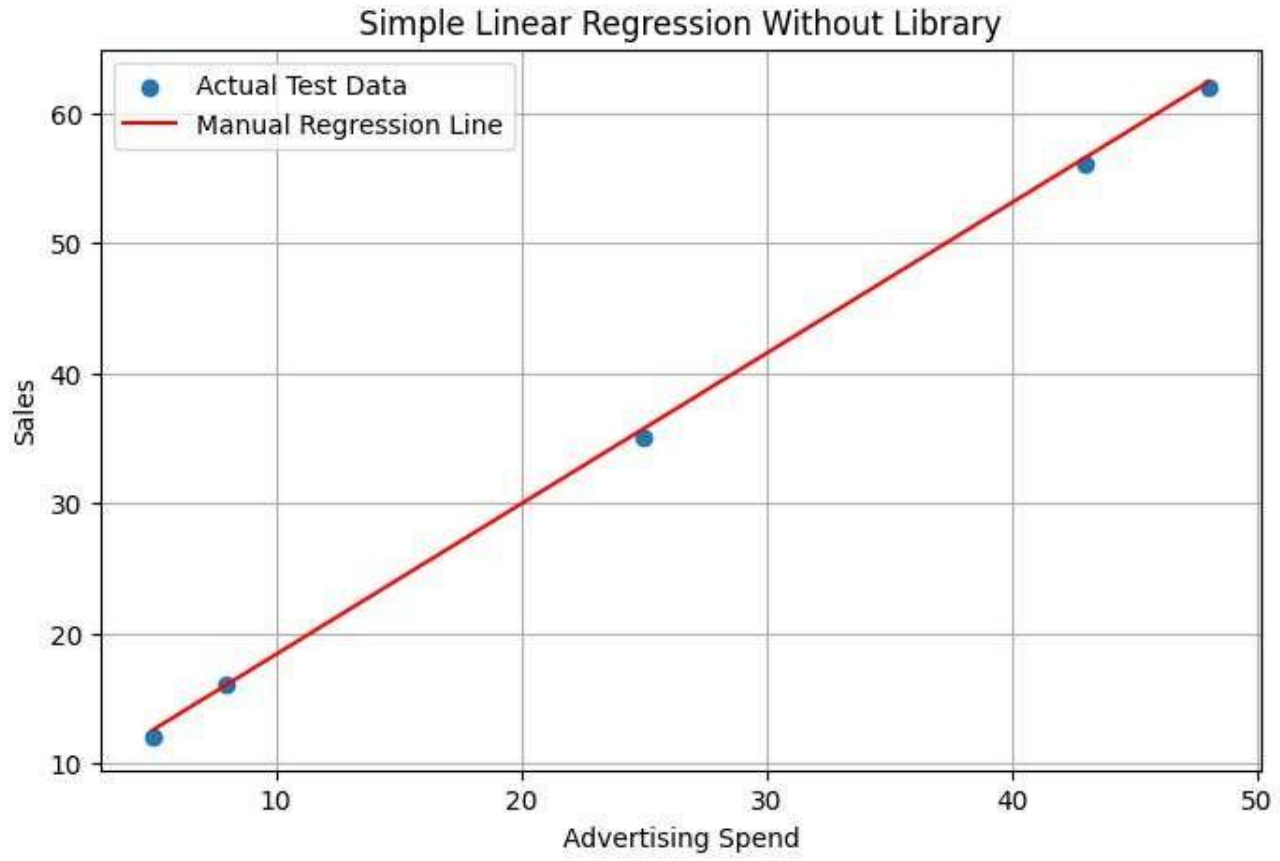
```
mse_manual = np.mean((y_test - y_adv_pred_manual) ** 2)
mae_manual = np.mean(np.abs(y_test - y_adv_pred_manual))
rmse_manual = np.sqrt(mse_manual)

ss_res = np.sum((y_test - y_adv_pred_manual) ** 2)
ss_tot = np.sum((y_test - np.mean(y_test)) ** 2)
r2_manual = 1 - ss_res / ss_tot

print("MAE:", mae_manual)
print("MSE:", mse_manual)
print("RMSE:", rmse_manual)
print("R²:", r2_manual)

x_sorted = np.sort(x_test)
plt.figure(figsize=(8, 5))
plt.scatter(x_test, y_test, label="Actual Test Data")
plt.plot(
    x_sorted,
    slope_manual * x_sorted + intercept_manual,
    color="red", label="Manual Regression Line"
)
plt.xlabel("Advertising Spend")
plt.ylabel("Sales")
plt.title("Simple Linear Regression Without Library")
plt.legend()
plt.grid(True)
plt.show()
```

```
MAE: 0.4600580794408639
MSE: 0.2673259873966593
RMSE: 0.5170357699392367
R²: 0.9993488747384142
```




```
from sklearn.datasets import load_diabetes

diabetes = load_diabetes(as_frame=True)
diabetes_df = diabetes.frame.copy()

print("First five rows:")
display(diabetes_df.head())

print("Dataset shape:", diabetes_df.shape)
print("Features:", diabetes.feature_names)

X_multi = diabetes_df[diabetes.feature_names]
y_multi = diabetes_df["target"]

X_multi_train, X_multi_test, y_multi_train, y_multi_test = (
    train_test_split(
        X_multi, y_multi,
        test_size=0.2,
        random_state=42
    )
)

multi_model = LinearRegression()
multi_model.fit(X_multi_train, y_multi_train)

y_multi_pred = multi_model.predict(X_multi_test)

print("Intercept:", multi_model.intercept_)

display(pd.DataFrame({
    "Feature": X_multi.columns,
```

```
    "Coefficient": multi_model.coef_
}))
```

```
First five rows:
```

	age	sex	bmi	bp	s1	s2	s3	s4	s5
0	0.038076	0.050680	0.061696	0.021872	-0.044223	-0.034821	-0.043401	-0.002592	0.019907
1	-0.001882	-0.044642	-0.051474	-0.026328	-0.008449	-0.019163	0.074412	-0.039493	-0.068332
2	0.085299	0.050680	0.044451	-0.005670	-0.045599	-0.034194	-0.032356	-0.002592	0.002861
3	-0.089063	-0.044642	-0.011595	-0.036656	0.012191	0.024991	-0.036038	0.034309	0.022688
4	0.005383	-0.044642	-0.036385	0.021872	0.003935	0.015596	0.008142	-0.002592	-0.031988

```
Dataset shape: (442, 11)
Features: ['age', 'sex', 'bmi', 'bp', 's1', 's2', 's3', 's4', 's5', 's6']
Intercept: 151.34560453985995
```

	Feature	Coefficient
0	age	37.904021
1	sex	-241.964362
2	bmi	542.428759
3	bp	347.703844
4	s1	-931.488846
5	s2	518.062277
6	s3	163.419983
7	s4	275.317902
8	s5	736.198859
9	s6	48.670657

Theory: Multiple Linear Regression Using a Library

Dataset: Diabetes dataset from Scikit-learn.

This dataset contains ten numerical input features, including age, sex, body mass index (BMI), blood pressure, and six blood-serum measurements. The target represents a quantitative measure of disease progression.

Multiple Linear Regression predicts one output using multiple independent variables.

Equation:

$$y = b_0 + b_1x_1 + b_2x_2 + \cdots + b_nx_n$$

Here, b_0 is the intercept, $b_1,\ldots,b_n$ are coefficients, and $x_1,\ldots,x_n$ are input features.

The `LinearRegression` class learns the coefficients from the training data. The model is evaluated on a separate test set.

```
mse_multi = mean_squared_error(y_multi_test, y_multi_pred)

print("MAE:", mean_absolute_error(y_multi_test, y_multi_pred))
print("MSE:", mse_multi)
```

```
print("RMSE:", np.sqrt(mse_multi))
print("R²:", r2_score(y_multi_test, y_multi_pred))
```

```
MAE: 42.79409467959994
MSE: 2900.1936284934814
RMSE: 53.85344583676593
R²: 0.4526027629719195
```

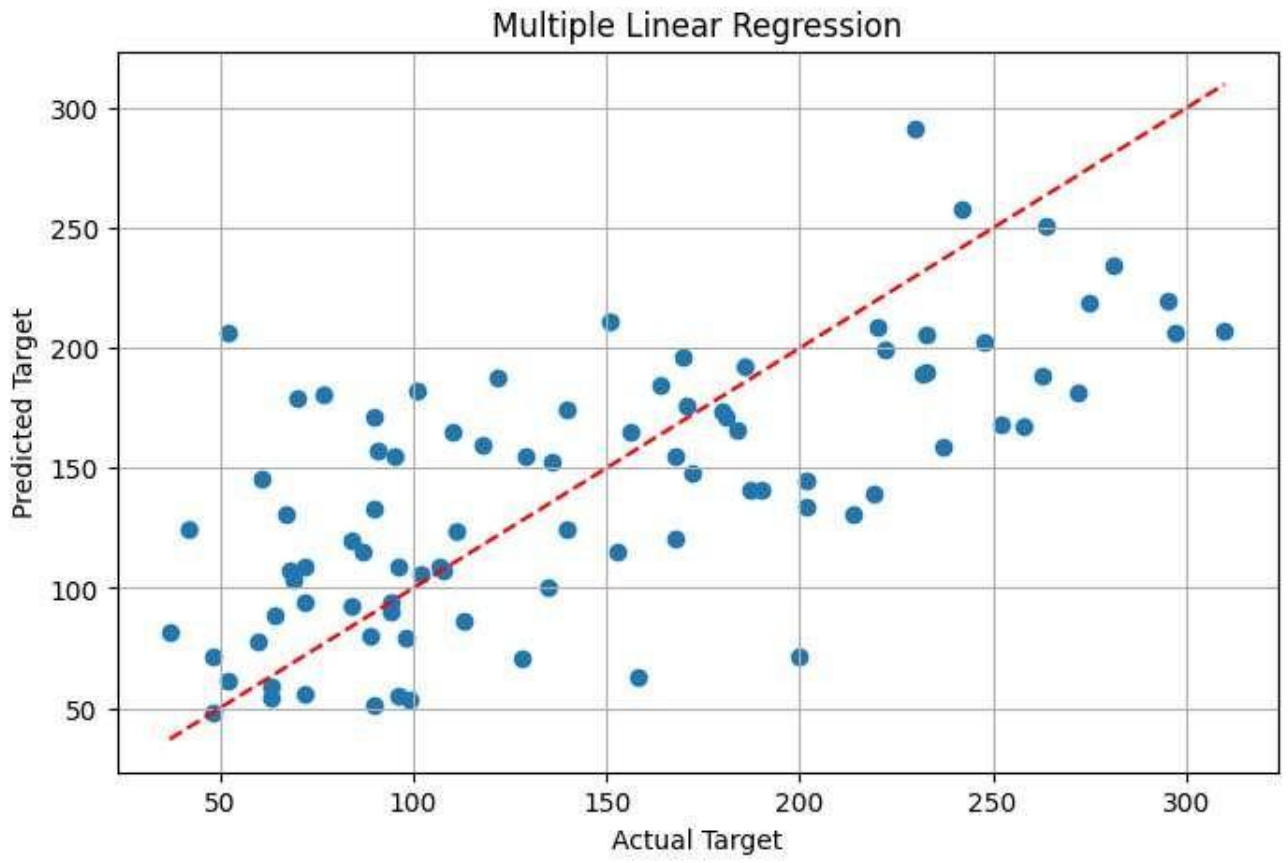
```
plt.figure(figsize=(8, 5))

plt.scatter(y_multi_test, y_multi_pred)

lower = min(y_multi_test.min(), y_multi_pred.min())
upper = max(y_multi_test.max(), y_multi_pred.max())

plt.plot([lower, upper], [lower, upper], "r--")

plt.xlabel("Actual Target")
plt.ylabel("Predicted Target")
plt.title("Multiple Linear Regression")
plt.grid(True)
plt.show()
```




```
# Use the same training and test data as Section 3
X_train_np = X_multi_train.to_numpy(dtype=float)
X_test_np = X_multi_test.to_numpy(dtype=float)

y_train_np = y_multi_train.to_numpy(dtype=float)
y_test_np = y_multi_test.to_numpy(dtype=float)

# Add intercept columns
X_train_b = np.column_stack([
    np.ones(X_train_np.shape[0]), X_train_np
])

X_test_b = np.column_stack([
```

```
    np.ones(X_test_np.shape[0]), X_test_np
])

# Solve least-squares regression manually
theta = np.linalg.lstsq(
    X_train_b, y_train_np, rcond=None
)[0]

y_multi_pred_manual = X_test_b @ theta

print("Intercept:", theta[0])
print("\nFeature coefficients:")

display(pd.DataFrame({
    "Feature": X_multi.columns,
    "Coefficient": theta[1:]
}))
```

```
Intercept: 151.3456045398604

Feature coefficients:
```

	Feature	Coefficient
0	age	37.904021
1	sex	-241.964362
2	bmi	542.428759
3	bp	347.703844
4	s1	-931.488846
5	s2	518.062277
6	s3	163.419983
7	s4	275.317902
8	s5	736.198859
9	s6	48.670657

Theory: Multiple Linear Regression Without a Regression Library

Dataset: Diabetes dataset — the same dataset used in Section 3.

In this implementation, the regression model is calculated manually using NumPy rather than the Scikit-learn `LinearRegression` class.

For multiple features, the least-squares solution can be written as:

$\theta = (X^T X)^{-1}X^T y$

In this experiment, `np.linalg.lstsq` is used to solve the least-squares problem more robustly than explicitly calculating a matrix inverse. A column of ones is added to represent the intercept. The learned coefficients are then used to predict the test-set values.

```
mse_multi_manual = mean_squared_error(
    y_test_np, y_multi_pred_manual
)

print("MAE:", mean_absolute_error(y_test_np, y_multi_pred_manual))
```

```
print("RMSE:", np.sqrt(mse_multi_manual))
print("R²:", r2_score(y_test_np, y_multi_pred_manual))
```

```
MAE: 42.79409467959991
MSE: 2900.193628493478
RMSE: 53.8534458367659
R²: 0.45260276297192015
```

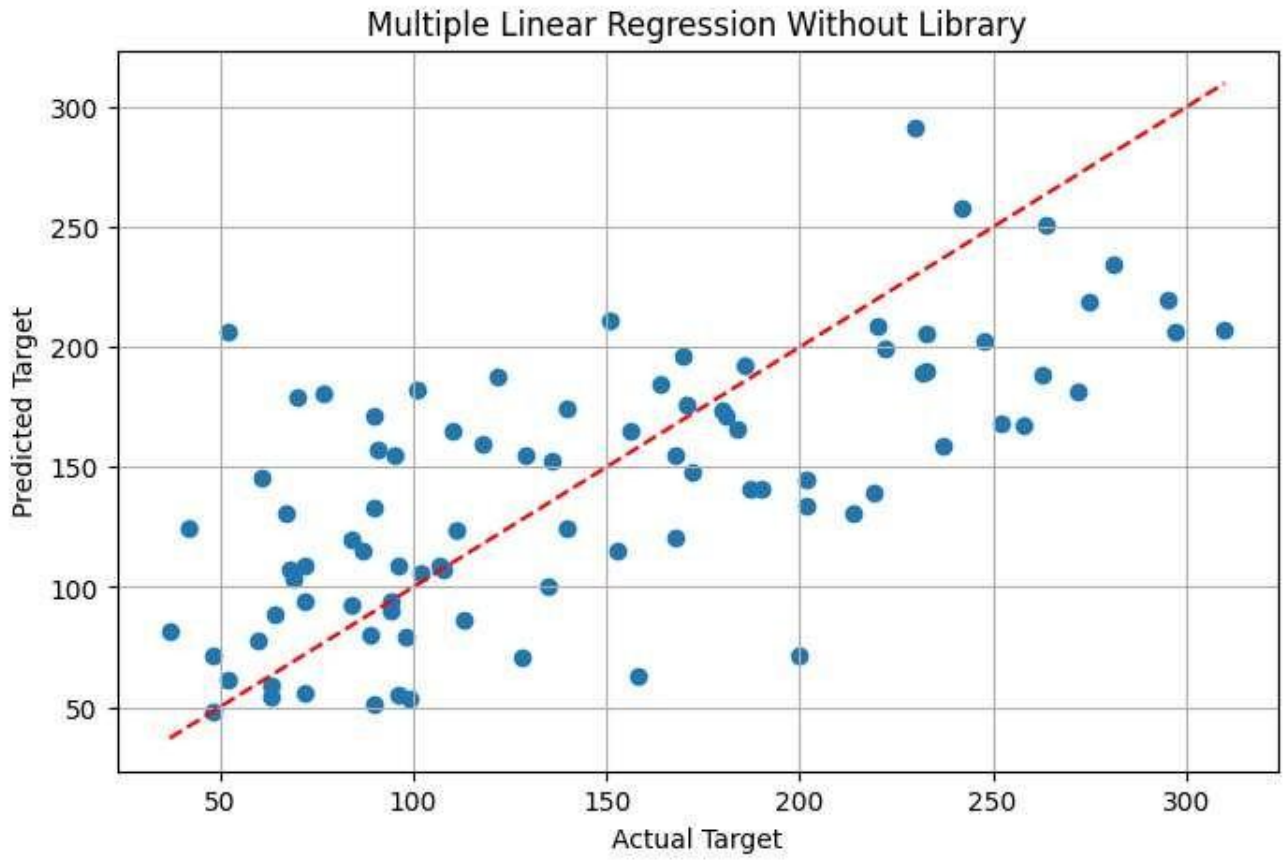
```
plt.figure(figsize=(8, 5))

plt.scatter(y_test_np, y_multi_pred_manual)

lower = min(y_test_np.min(), y_multi_pred_manual.min())
upper = max(y_test_np.max(), y_multi_pred_manual.max())

plt.plot([lower, upper], [lower, upper], "r--")

plt.xlabel("Actual Target")
plt.ylabel("Predicted Target")
plt.title("Multiple Linear Regression Without Library")
plt.grid(True)
plt.show()
```




```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import make_pipeline

X_poly = diabetes_df[["bmi"]]
y_poly = diabetes_df["target"]

X_poly_train, X_poly_test, y_poly_train, y_poly_test = (
    train_test_split(
        X_poly, y_poly,
        test_size=0.2,
        random_state=42
    )
)
```

```
poly_model = make_pipeline(
    PolynomialFeatures(degree=2),
    LinearRegression()
)

poly_model.fit(X_poly_train, y_poly_train)
y_poly_pred = poly_model.predict(X_poly_test)

mse_poly = mean_squared_error(y_poly_test, y_poly_pred)

print("Polynomial Degree: 2")
print("MAE:", mean_absolute_error(y_poly_test, y_poly_pred))
print("MSE:", mse_poly)
print("RMSE:", np.sqrt(mse_poly))
print("R²:", r2_score(y_poly_test, y_poly_pred))
```

```
Polynomial Degree: 2
MAE: 52.38391176015265
MSE: 4085.0254808716318
RMSE: 63.914204061942534
R²: 0.2289715971205668
```

Theory: Polynomial Regression

Dataset: Diabetes dataset — the same dataset used for Multiple Linear Regression.

Polynomial Regression models a potentially curved relationship between an input feature and the output by adding powers of the input feature.

For degree 2, the equation is:

(y = b_0 + b_1x + b_2x^2)

In this experiment, BMI is selected as the single input feature and disease progression is the target. The `PolynomialFeatures` class generates the squared BMI feature, and `LinearRegression` estimates the coefficients.

Polynomial Regression can capture some nonlinear patterns that a straight line may not represent.

```
bmi_values = np.linspace(
    X_poly["bmi"].min(),
    X_poly["bmi"].max(),
    200
)

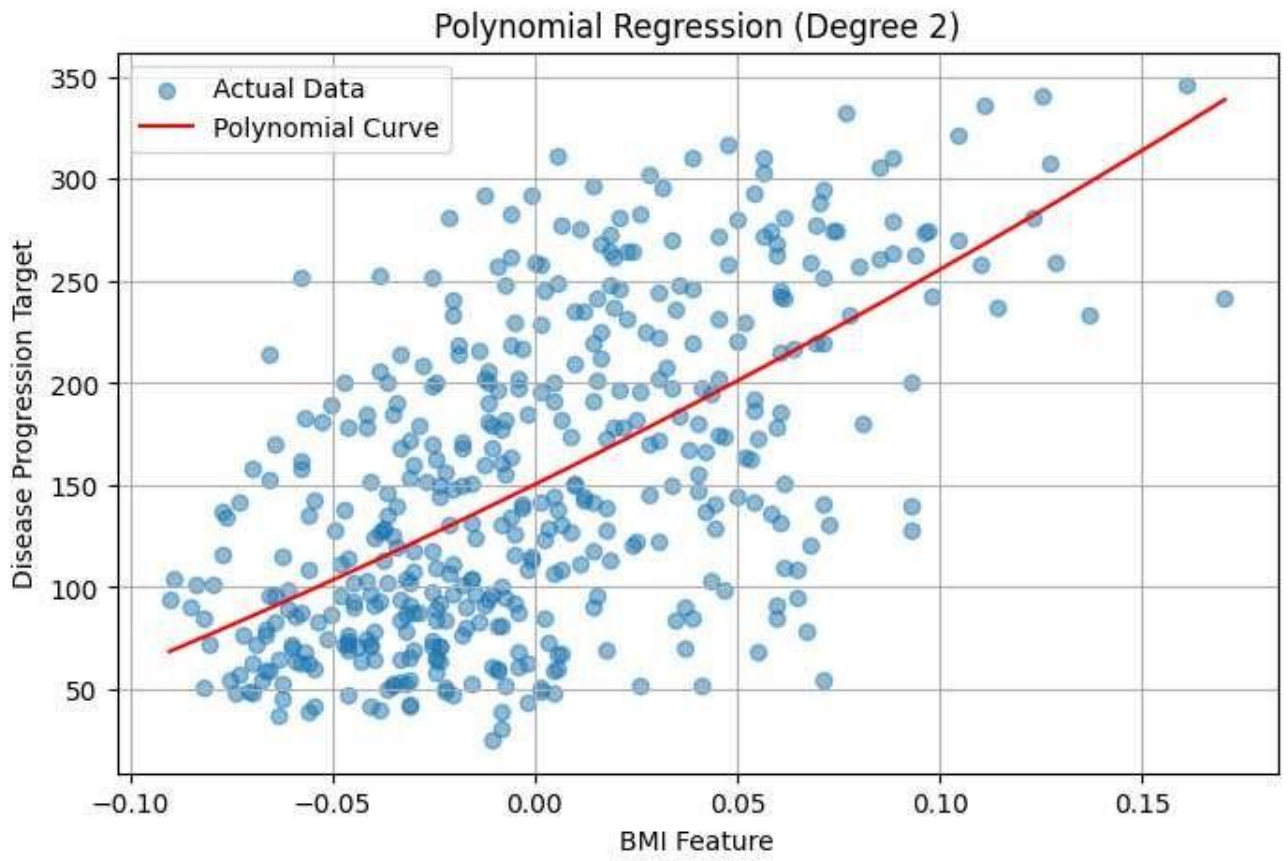
bmi_frame = pd.DataFrame({"bmi": bmi_values})
curve_predictions = poly_model.predict(bmi_frame)

plt.figure(figsize=(8, 5))

plt.scatter(
    X_poly["bmi"], y_poly,
    alpha=0.5, label="Actual Data"
)

plt.plot(
    bmi_values, curve_predictions,
    color="red", label="Polynomial Curve"
)
```

```
plt.xlabel("BMI Feature")
plt.ylabel("Disease Progression Target")
plt.title("Polynomial Regression (Degree 2)")
plt.legend()
plt.grid(True)
plt.show()
```




```
# Use the same Diabetes dataset
X_gd = diabetes_df[diabetes.feature_names].to_numpy(dtype=float)
y_gd = diabetes_df["target"].to_numpy(dtype=float)

X_gd_train, X_gd_test, y_gd_train, y_gd_test = train_test_split(
    X_gd, y_gd,
    test_size=0.2,
    random_state=42
)

# Standardize features using training data only
X_mean = X_gd_train.mean(axis=0)
X_std = X_gd_train.std(axis=0)
X_std[X_std == 0] = 1

X_gd_train_scaled = (X_gd_train - X_mean) / X_std
X_gd_test_scaled = (X_gd_test - X_mean) / X_std

# Standardize target using training data only
y_mean = y_gd_train.mean()
y_std = y_gd_train.std()

y_gd_train_scaled = (y_gd_train - y_mean) / y_std

# Add intercept
X_gd_train_b = np.column_stack([
    np.ones(len(X_gd_train_scaled)),
    X_gd_train_scaled
])
```

```
X_gd_test_b = np.column_stack([
    np.ones(len(X_gd_test_scaled)),
    X_gd_test_scaled
])

# Initialize parameters
theta_gd = np.zeros(X_gd_train_b.shape[1])

learning_rate = 0.03
iterations = 3000
m = len(y_gd_train_scaled)

cost_history = []

# Gradient Descent
for i in range(iterations):
    predictions = X_gd_train_b @ theta_gd
    errors = predictions - y_gd_train_scaled

    cost = np.sum(errors ** 2) / (2 * m)
    cost_history.append(cost)

    gradients = (X_gd_train_b.T @ errors) / m
    theta_gd = theta_gd - learning_rate * gradients

# Predict and convert to the original target scale
y_gd_pred_scaled = X_gd_test_b @ theta_gd
y_gd_pred = y_gd_pred_scaled * y_std + y_mean

print("Gradient Descent Completed")
print("Iterations:", iterations)
print("Final Training Cost:", cost_history[-1])
```

```
Gradient Descent Completed
Iterations: 3000
Final Training Cost: 0.2365320135134553
```

Theory: Gradient Descent

Dataset: Diabetes dataset — the same dataset used for Multiple Linear Regression and Polynomial Regression.

Gradient Descent is an optimization algorithm that learns model parameters by repeatedly reducing a cost function. In this experiment, it is implemented from scratch using NumPy.

The update rule is:

(\theta := \theta - \alpha \frac{1}{m}X^T(X\theta-y))

Where:

- (\theta) represents the model parameters.
- (\alpha) is the learning rate.
- (m) is the number of training samples.
- (X) is the feature matrix.
- (y) contains the target values.

The input features and target are standardized to help the algorithm converge. The cost is recorded at each iteration, and the final predictions are evaluated on the test dataset.

```
mse_gd = mean_squared_error(y_gd_test, y_gd_pred)

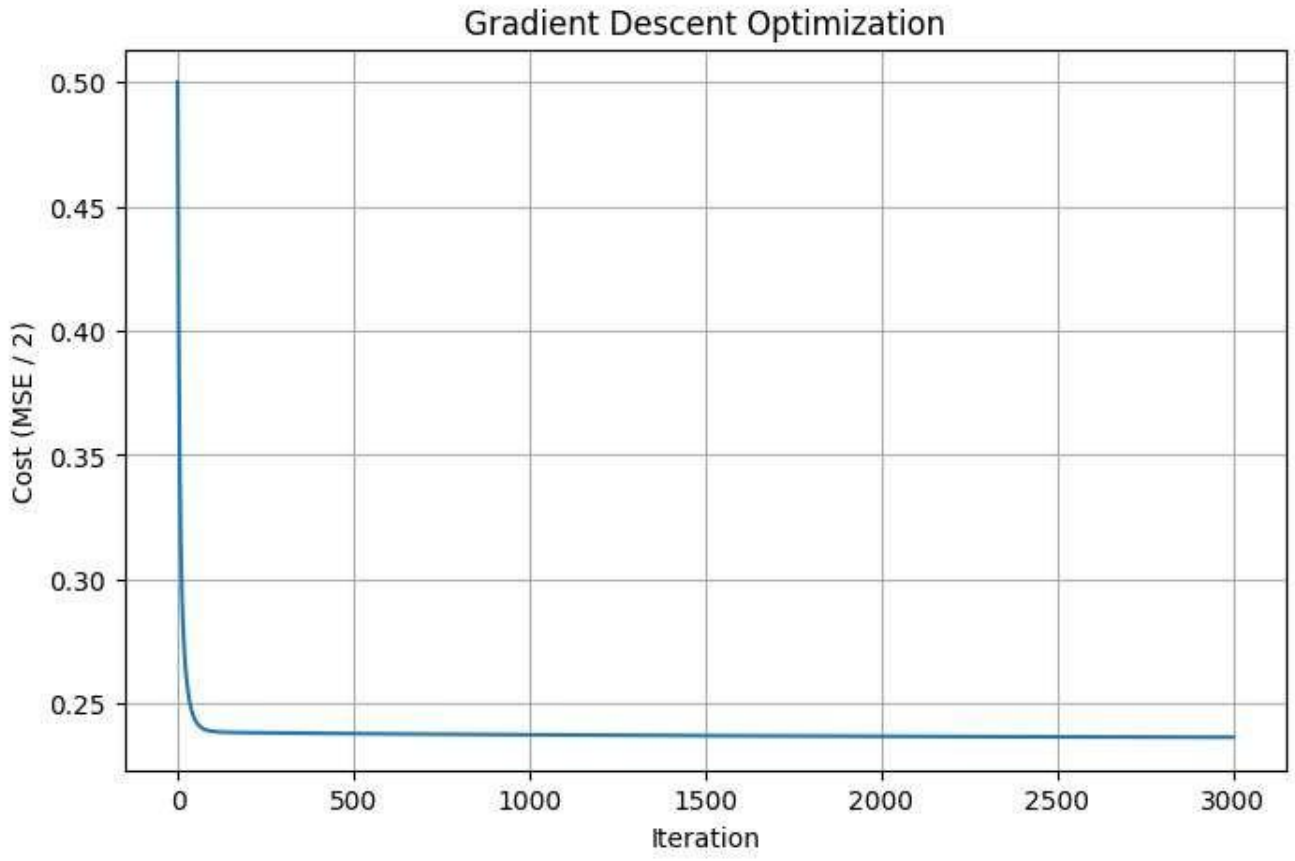
print("MAE:", mean_absolute_error(y_gd_test, y_gd_pred))
print("MSE:", mse_gd)
print("RMSE:", np.sqrt(mse_gd))
print("R²:", r2_score(y_gd_test, y_gd_pred))
```

```
MAE: 42.83157062034938
MSE: 2889.288538472775
RMSE: 53.75210264234112
R²: 0.45466104490462544
```

```
plt.figure(figsize=(8, 5))

plt.plot(cost_history)

plt.xlabel("Iteration")
plt.ylabel("Cost (MSE / 2)")
plt.title("Gradient Descent Optimization")
plt.grid(True)
plt.show()
```




```
comparison = pd.DataFrame([
    {
        "Model": "Simple Linear Regression (Library)",
        "Dataset": "Advertising vs Sales",
        "MAE": mean_absolute_error(y_adv_test, y_adv_pred),
        "MSE": mean_squared_error(y_adv_test, y_adv_pred),
        "RMSE": np.sqrt(mean_squared_error(y_adv_test, y_adv_pred)),
        "R2": r2_score(y_adv_test, y_adv_pred)
    },
    {
        "Model": "Simple Linear Regression (Manual)",
        "Dataset": "Advertising vs Sales",
        "MAE": mae_manual,
```

```
        "MSE": mse_manual,
        "RMSE": rmse_manual,
        "R2": r2_manual
    },
    {
        "Model": "Multiple Linear Regression (Library)",
        "Dataset": "Diabetes",
        "MAE": mean_absolute_error(y_multi_test, y_multi_pred),
        "MSE": mean_squared_error(y_multi_test, y_multi_pred),
        "RMSE": np.sqrt(mean_squared_error(y_multi_test, y_multi_pred)),
        "R2": r2_score(y_multi_test, y_multi_pred)
    },
    {
        "Model": "Multiple Linear Regression (Manual)",
        "Dataset": "Diabetes",
        "MAE": mean_absolute_error(y_test_np, y_multi_pred_manual),
        "MSE": mse_multi_manual,
        "RMSE": np.sqrt(mse_multi_manual),
        "R2": r2_score(y_test_np, y_multi_pred_manual)
    },
    {
        "Model": "Polynomial Regression",
        "Dataset": "Diabetes (BMI)",
        "MAE": mean_absolute_error(y_poly_test, y_poly_pred),
        "MSE": mse_poly,
        "RMSE": np.sqrt(mse_poly),
        "R2": r2_score(y_poly_test, y_poly_pred)
    },
    {
        "Model": "Gradient Descent",
        "Dataset": "Diabetes",
        "MAE": mean_absolute_error(y_gd_test, y_gd_pred),
        "MSE": mse_gd,
        "RMSE": np.sqrt(mse_gd),
        "R2": r2_score(y_gd_test, y_gd_pred)
    }
])

display(comparison.round(4))
```

	Model	Dataset	MAE	MSE	RMSE	R2
0	Simple Linear Regression (Library)	Advertising vs Sales	0.4601	0.2673	0.5170	0.9993
1	Simple Linear Regression (Manual)	Advertising vs Sales	0.4601	0.2673	0.5170	0.9993
2	Multiple Linear Regression (Library)	Diabetes	42.7941	2900.1936	53.8534	0.4526
3	Multiple Linear Regression (Manual)	Diabetes	42.7941	2900.1936	53.8534	0.4526
4	Polynomial Regression	Diabetes (BMI)	52.3839	4085.0255	63.9142	0.2290
5	Gradient Descent	Diabetes	42.8316	2889.2885	53.7521	0.4547

Theory: Performance Evaluation

The models are compared using four evaluation metrics.

- **MAE:** Average absolute prediction error.
- **MSE:** Average squared prediction error.
- **RMSE:** Square root of the mean squared error.

- **R^2:** Indicates how much variation in the target is explained by the model.

Lower MAE, MSE, and RMSE generally indicate smaller prediction errors. A higher R^2 generally indicates a better fit. Since the simple regression predicts sales and the other models predict the Diabetes target, their raw error values should not be compared directly across datasets.

Comparison of Regression Model Evaluation Metrics

To compare the performance of regression models, we use MAE, MSE, RMSE, and R^2. These metrics measure the difference between actual and predicted values and help us determine how well a model performs.

1. Mean Absolute Error (MAE)

MAE calculates the average absolute difference between actual and predicted values. A lower MAE indicates better predictions because the average error is smaller. It is easy to understand and is less sensitive to large errors than MSE.

2. Mean Squared Error (MSE)

MSE calculates the average of the squared differences between actual and predicted values. A lower MSE indicates better performance. Because errors are squared, large prediction errors receive more penalty than small errors.

3. Root Mean Squared Error (RMSE)

RMSE is the square root of MSE. A lower RMSE indicates better prediction accuracy. Its main advantage is that it is expressed in the same units as the target variable, making it easier to interpret than MSE.

4. R-squared (R^2)

R^2 measures how much of the variation in the target variable is explained by the regression model. A higher R^2 generally indicates a better fit. An R^2 of 1 represents perfect predictions, 0 means the model performs like predicting the test-set mean under the standard definition, and a negative value means it performs worse than that baseline.

Which Model Is Better?

A model is generally considered better when it has:

- **Lower MAE:** smaller average absolute prediction errors.
- **Lower MSE:** smaller squared prediction errors.
- **Lower RMSE:** smaller prediction errors in the target's units.
- **Higher R^2:** more variation explained by the model.

No single metric should be used alone. MAE is useful for understanding typical error, MSE penalizes large errors more strongly, RMSE expresses error in the original target units, and R^2 measures how well the model explains variation.

When comparing models, use the same test dataset and target variable. In this experiment, the two simple regression models predict sales, while the multiple, polynomial, and gradient descent models predict the Diabetes dataset's target. Therefore, compare models within the same dataset rather than comparing their raw error values across different datasets.

Conclusion: The best model generally combines low MAE, MSE, and RMSE with high R^2 on unseen test data. The final choice should also consider the problem's requirements and whether the model

generalizes well to new data.

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Use the same Diabetes dataset
X = diabetes_df[diabetes.feature_names].to_numpy(dtype=float)
y = diabetes_df["target"].to_numpy(dtype=float)

# Split the data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Scale features using training data only
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Locally Weighted Regression function
def locally_weighted_regression(X_train, y_train, X_test, tau=1.0):
    predictions = []

    X_train_bias = np.c_[np.ones(len(X_train)), X_train]
    X_test_bias = np.c_[np.ones(len(X_test)), X_test]

    for x in X_test_bias:
        distances = np.sum((X_train_bias[:, 1:] - x[1:]) ** 2, axis=1)

        weights = np.exp(-distances / (2 * tau ** 2))
        W = np.diag(weights)

        A = X_train_bias.T @ W @ X_train_bias
        b = X_train_bias.T @ W @ y_train

        theta = np.linalg.pinv(A) @ b
        predictions.append(x @ theta)

    return np.array(predictions)

# Train and predict
y_pred_lwr = locally_weighted_regression(
    X_train_scaled, y_train, X_test_scaled, tau=1.0
)

# Evaluate the model
mae_lwr = mean_absolute_error(y_test, y_pred_lwr)
mse_lwr = mean_squared_error(y_test, y_pred_lwr)
rmse_lwr = np.sqrt(mse_lwr)
r2_lwr = r2_score(y_test, y_pred_lwr)

print("Locally Weighted Regression Results")
print("MAE:", round(mae_lwr, 4))
print("MSE:", round(mse_lwr, 4))
print("RMSE:", round(rmse_lwr, 4))
print("R2 Score:", round(r2_lwr, 4))
```

```
Locally Weighted Regression Results
MAE: 42.328
MSE: 2944.819
RMSE: 54.2662
R2 Score: 0.4442
```

Locally Weighted Regression (LWR) is a non-parametric regression algorithm that predicts a target value by giving greater weight to training samples that are closer to the point being predicted. It fits a local linear model for each prediction instead of learning one fixed model for the entire dataset.

Objective: To apply Locally Weighted Regression to the Diabetes dataset and evaluate its performance using MAE, MSE, RMSE, and (R^2).

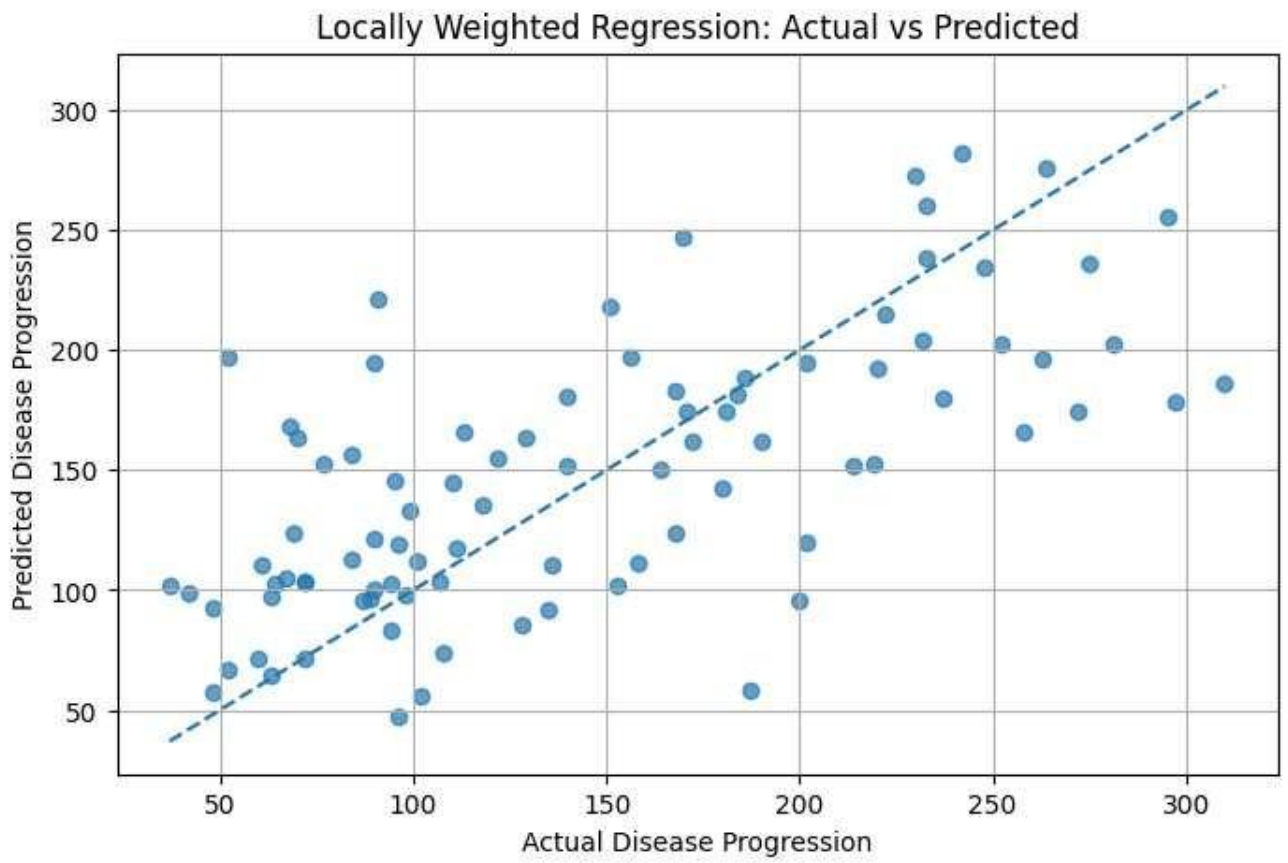
```
plt.figure(figsize=(8, 5))

plt.scatter(y_test, y_pred_lwr, alpha=0.7)

lower = min(y_test.min(), y_pred_lwr.min())
upper = max(y_test.max(), y_pred_lwr.max())

plt.plot([lower, upper], [lower, upper], linestyle="--")

plt.xlabel("Actual Disease Progression")
plt.ylabel("Predicted Disease Progression")
plt.title("Locally Weighted Regression: Actual vs Predicted")
plt.grid(True)
plt.show()
```




```
{
    "Model": "Locally Weighted Regression",
    "Dataset": "Diabetes",
    "MAE": mae_lwr,
    "MSE": mse_lwr,
    "RMSE": rmse_lwr,
    "R²": r2_lwr
```

```
}
```

```
{'Model': 'Locally Weighted Regression',
 'Dataset': 'Diabetes',
 'MAE': 42.32797807780978,
 'MSE': 2944.8190461936615,
 'RMSE': np.float64(54.26618695093347),
 'R²': 0.44417993557504953}
```

Conclusion

In this assignment, I implemented different regression techniques, including Simple Linear Regression, Multiple Linear Regression, Polynomial Regression, Gradient Descent, and Locally Weighted Regression. I used the Advertising dataset for Simple Linear Regression and the Diabetes dataset for the other regression techniques. I evaluated the models using MAE, MSE, RMSE, and R^2 scores to understand their performance. This assignment helped me understand how regression models work, how predictions are made, and how different techniques can be compared to solve prediction problems.